

CONTENTS

Kurumdan öğreneceklerim	
Nasıl kullanılır	
BÖLÜM 1 — Kesinlikle sorulması gerekenler	
1.1 ★ Veri tabanı isimlendirme kuralları — EN ÖNEMLİSİ	
1.2 Kod nereye gönderilecek	
1.3 Teslim tarihi	
BÖLÜM 2 — DevOps sınırı: kim ne yapıyor	
2.1 Migration'ı canlıda kim çalıştırıyor	
2.2 Gizli değerleri kim, nereye giriyor	
2.3 Bir sürüm bozarsa geri almayı kim yapıyor	
BÖLÜM 3 — Kapsam soruları (istenirse süre isteyeceğin şeyler)	
BÖLÜM 4 — Sorulmayacaklar (ben karar veriyorum)	

Kurumdan öğreneceklerim

Bu dosya, İzmir Büyükşehir’e soracağın soruların listesi. Her sorunun yanında **neden sorduğun** ve **cevaba göre ne değişeceği** yazılı — böylece karşı taraf “bu niye soruyor” demez, sen de cevabı alınca ne yapacağını bilirsin.

🚫 **Buradaki hiçbir soru mühendislik sorusu değildir.** Hepsi yalnızca kurumun bilebileceği **olgular**. Teknik kararları ben veriyorum, sana gerekçesiyle onaylatıyorum.

Nasıl kullanılır

Durum	Ne yap
Cevabı aldın	Satırın Cevap kolonunu doldur
Cevap “bilmiyorum” geldi	⚠️ Varsayım kolonunu doldur — ben o varsayımla ilerlerim
Soru gereksiz çıktı	Sil, ama neden gereksizdi yaz

★ Cevaplar geldikçe `docs/project/PRD.md` ve `docs/project/altyapi-durumu.md` dosyalarına işlenir. Bu dosya kurulum bitince silinir.

BÖLÜM 1 — Kesinlikle sorulması gerekenler

Bunlar cevaplanmadan **kod yazılmaya başlanabilir**, ama teslimden önce mutlaka netleşmeli.

1.1 ★ Veri tabanı isimlendirme kuralları — EN ÖNEMLİSİ

Soru: "Ödevin 11. maddesinde 'veri tabanı tasarımı **kurum tarafından iletilen** veri tabanı geliştirme ve isimlendirme kurallarına uygun olmalıdır' yazıyor. Böyle bir doküman var mı, alabilir miyim?"

Neden soruyorum	Ödev metni böyle bir dokümandan bahsediyor ama eklememiş . Varsa ve uymazsak, teslim doğrudan kural ihlali sayılır
Cevaba göre ne değişir	Tablo adları (<code>work_order</code> mı <code>WorkOrder</code> mı), kolon adları, birincil anahtar biçimi, tarih alanı adları — tüm veri modeli
Cevap gelmezse	PostgreSQL yaygın pratiği: <code>snake_case</code> tablo/kolon, çoğul tablo adı, <code>id</code> birincil anahtar, <code>created_at</code> / <code>updated_at</code> . PRD → Varsayımlar'a yazılır
Cevap	(doldurulacak)

⚠ **Bu soruyu ilk gün sor.** Cevap sonradan gelirse tüm migration'lar yeniden yazılır.

1.2 Kod nereye gönderilecek

Soru: "Bu ödev için kodu nereye göndermemi istersiniz — GitHub mı, kurumun GitLab'ı mı? GitLab ise hesap ve adres verilecek mi?"

Neden soruyorum	Ödev §30 Pull Request veya Merge Request istiyor. PR/MR yalnızca bir barındırma servisinde olur; git tek başına üretemez
Cevaba göre ne değişir	Hangi CI dosyası birincil olur (<code>.github/workflows/ci.yml</code> mi <code>.gitlab-ci.yml</code> mi) ve teslim linkini nereden vereceğin
Cevap gelmezse	GitHub'da başlarım. ★ İkisi de hazır olacak — CI adımları <code>pnpm ci:verify</code> içinde, iki platform dosyası da onu çağırıyor. GitLab adresi sonra gelirse ikinci remote eklenir, ek iş çıkmaz
Cevap	(doldurulacak)

1.3 Teslim tarihi

Soru: "Teslim için son tarih nedir? Teknik sunum ne zaman planlanıyor?"

Neden soruyorum	Ödev metninde hiçbir yerde tarih yok (795 satır tarandı). Kapsam kararları buna göre verilir
Cevaba göre ne değişir	Süre darsa: mobil, dosya yükleme gibi ödevde istenmeyen maddeler kapsam dışı kalır. Bolsa eklenebilir
Cevap gelmezse	~10 iş günü hedefiyle ilerlenir
Cevap	(doldurulacak)

BÖLÜM 2 — DevOps sınırı: kim ne yapıyor

Bu üç soru, “**senin işin nerede bitiyor**” sorusunun cevabı. Kurumda bu sınır yazılı olmadığı için en çok karışıklık buradan çıkar. Cevaplar `altyapi-durumu.md`’ye yazılır — kodda görünmezler ve altı ay sonra kimse hatırlamaz.

2.1 Migration’ı canlıda kim çalıştırıyor

Soru: “Veritabanı şema değişikliklerini (migration) canlıda kim çalıştırıyor — uygulama açılışta kendisi mi yapıyor, yoksa DevOps ayrı bir adımda mı koşturuyor?”

Bu ne demek — açıklaması:

Veritabanına yeni bir tablo veya kolon eklediğimde, bu değişikliği **canlı veritabanına da uygulamak** gerekir. Buna *migration* denir. İki yol var:

Yol	Nasıl	Artısı	Eksisi
A) Uygulama kendisi	Konteyner açılırken <code>prisma migrate deploy</code> çalışır	Ekstra adım yok	⚠ İki kopya aynı anda açılırsa ikisi birden çalıştırmaya kalkar
B) DevOps ayrı adımda	Yayına almadan önce elle veya hatta bir adım olarak	Kontrollü, geri alınabilir	Koordinasyon gerekir
Cevaba göre ne değişir	<code>Dockerfile</code> ’ın başlangıç komutu ve teslim belgesindeki yayına alma sırası		
Cevap gelmezse	B varsayılır (kurumsal ortamda yaygın olan). Migration komutu README’de ayrı bir adım olarak belgelenir		
Cevap	(doldurulacak)		

2.2 Gizli değerleri kim, nereye giriyor

Soru: "Veritabanı şifresi, JWT anahtarı gibi gizli değerleri canlıda kim giriyor ve nereye — bir panele mi, sunucudaki dosyaya mı, yoksa kurumun bir gizli değer yönetim sistemi mi var?"

Bu ne demek — açıklaması:

Uygulamanın çalışması için şifre ve anahtarlara ihtiyacı var. Bunlar **koda yazılmaz** (git geçmişisi silinmez, bir kez girerse orada kalır). Canlıda bir şekilde uygulamaya verilmesi gerekir.

Cevaba göre ne değişir	<code>.env.example</code> dosyasının nasıl yazılacağı ve DevOps'a verilecek talimat
Cevap gelmezse	Standart yol: <code>.env.example</code> 'da adları ve ne işe yaradıkları belgelenir, değerleri DevOps girer. Bu her durumda çalışır
Cevap	(doldurulacak)

2.3 Bir sürüm bozarsa geri almayı kim yapıyor

Soru: "Yayına alınan bir sürüm sorun çıkarırsa geri alma (rollback) kararını kim veriyor ve nasıl yapılıyor?"

Bu ne demek — açıklaması:

Yeni sürüm canlıya çıktı ve bir şey bozuldu. Eski sürüme dönmek gerekiyor. ⚠ **Zor kısmı veritabanı:** kod geri alınabilir ama **migration geri alınamaz** — silinen bir kolon geri gelmez.

Cevaba göre ne değişir	Migration'ları geriye uyumlu yazma zorunluluğu. Örneğin bir kolonu silmek yerine önce kullanımdan kaldırıp bir sonraki sürümde silmek
Cevap gelmezse	★ Her durumda geriye uyumlu yazarım — bu zaten best practice. Kolon silme/yeniden adlandırma iki aşamaya bölünür
Cevap	(doldurulacak)

BÖLÜM 3 — Kapsam soruları (istenirse süre isteyeceğin şeyler)

Bunlar **ödevde istenmiyor**. Sorulmazsa gündeme getirmene gerek yok. Ama “şu da olsa iyi olurdu” denirse cevabın hazır olsun.

Konu	Ödevde var mı	Eklenirse ne olur
Mobil uygulama (Expo)	🚫 Yok	⚠️ Ek süre gerekir. Mimari hazır (aynı API), ama ekranlar, mağaza süreci ve test ayrı iş
Fotoğraf / dosya eki	🚫 Yok — yalnızca yorum isteniyor (§39, §91)	Depolama servisi + dosya doğrulama + boyut sınırı + KVKK (fotoğrafta insan olabilir). ★ Veri modelinde yeri hazır bırakılacak , eklenmesi kolay olsun
Gerçek e-posta / SMS	🚫 Yok — sistem içi bildirim isteniyor (§16)	Sağlayıcı hesabı + alan adı doğrulama gerekir
Gerçek personel entegrasyonu	🚫 Yok	Sahte veri kullanılacak (§5.1 gerçek veriyi zaten yasaklıyor)
Çok dillilik	🚫 Yok	Tek dil: Türkçe

★ Bunların hepsi PRD → “Kapsam dışı” bölümüne gerekçesiyle yazılacak.

Yazmak zayıflık değil, **kapsama hâkim olmak** demektir — ödev §31 zaten “bilinen eksikler”i soruyor.

BÖLÜM 4 — Sorulmayacaklar (ben karar veriyorum)

Bunları kuruma sorma. Cevabı kurumun bilmesi beklenmez ve zaten karara bağlandı — hepsinin gerekçesi `proje-teknoloji-ve-plan.md` içinde:

Konu	Karar	Gerekçe nerede
Hangi teknolojiler	Next.js + NestJS + Prisma + PostgreSQL + Zod + BullMQ	BÖLÜM A, C
Mimari	Modüler monolit + Clean Architecture	E.1
SLA süreleri ve takvim	Kritik 3sa (7/24) · Yüksek 8sa · Normal 24sa · Düşük 72sa (mesai)	E.4
İş emri numarası biçimi	<code>IE-2026-000148</code> — yıl + artan sayaç	E.4 civarı
Sayfalama	Offset (sayfa numarası gösterildiği için)	E.10
Mapping	Zod şeması + Prisma <code>select</code> — kütüphane yok	E.6
Test stratejisi	Vitest + Testcontainers + dependency-cruiser + Playwright	C.7, C.11
Veri modeli tasarımı	★ Ödev açıkça "tarafınızdan tasarlanmalıdır" diyor (§11)	E.9

🚫 **§11'in son cümlesi önemli:** "Hazır bir şemayı veya bu çalışmayla ilgisiz mevcut bir projeyi uyarlamak yerine, verilen iş ihtiyacına göre veri modeli oluşturmanız beklenmektedir."

Yani **başka bir yerden şema kopyalamak açıkça yasaklanmış**. Veri modelini ödevin iş kurallarından türeteceğiz.